ExtremONet: Extreme-Learning-based Neural Operator for identifying dynamical systems.

Paper #123

Abstract. The DeepONet, first described by L. Lu, P. Jin, and G. E. Karniadakis in 2019, made a noticeable impact on Deep Learning research. The model they described is able to learn maps between function spaces, unlike Neural Networks which learn maps between vector spaces. The DeepONet is based on the Universal Approximation Theorem for Operator (UATFO), which has some crucial differences to the Universal Approximation Theorem for Neural Networks. It was shown that this model is able to learn and generalize the operators that map a time point to the solutions of differential equations. Because the DeepONet architecture requires two Neural Networks, it is computationally expensive to train. In this paper, we introduce the *ExtremONet*: an Extreme-Learning-Machine based variation of the DeepONet. Extreme-learning machines are universal function approximators, generally trained via Ridge Regression. We extended this notion to operator learning by adjusting the readout form of the DeepONet, resulting in one-step learning of maps between function spaces. We showed that ExtremONets are able to outperform small, moderately-sized, and large DeepONets on 2 out of 4 problem settings that were considered, while training between 100x-10 000x faster. This paper concludes with an exploration of out-of-distribution generalization.

1 Introduction

Machine Learning has been a valuable tool for the exact sciences [1, 2, 3, 4]. In more recent years, personal computing systems have become capable of training multi-million-parameter Neural Networks in a relatively short amount of time, allowing researchers to use and develop advanced ML models without relying on enterprise-grade hardware [5]. A notable example is the principle of Physics Informed Neural Networks (PINNs), first described in 2019 by [6] demonstrated that including knowledge of the underlying dynamics of a system in the loss function increases performance, and drastically reduces the amount of data necessary to generalize. PINNs bridged the gap between data-driven and model-driven approaches. In its wake a new field of study arose [7]; Neural Operators, which was established by the DeepONet architecture [8], can be viewed as a generalization of Neural Networks, where the model learns a map between function spaces instead of between vector spaces. Where PINNs used information about the system to aid optimization, Neural Operators encode mathematical structure into the Neural Network architecture itself, creating an application-specific method. An example of an operator in the context of dynamics is finding the solution to some initial value problem of a differential equation $d_t y = f(y,t) + u(y,t)$ where $y(a) = y_a$. If the differential equation is first order, and not dependent on y itself, then we define such an operator as the anti-derivative: $G(u(\cdot))(t) \to y(t) = \int_0^t f(t') + u(t')dt'$. This does not have a simple integral solution if the previous assumptions do not apply. The goal of a Neural Operator $\mathscr{G}$ in the context of solving an initial value problem is now to approximate[1] $G(u(\cdot))(t)$, which cannot be done with traditional Neural Networks as they can only learn a single $u(\cdot)$ per network instance, nor do they generalize well when information regarding $u(\cdot)$ is included in the input. In this work, we present a novel Extreme-Learning-Machines-based Neural Operator. The literature regarding approximating operators has not yet been extended to random-network-architectures. We experimentally show that random networks can be used to approximate operators.

2 Related work

Dynamics

Neural Networks have proven useful for modelling Non-Linear Dynamics [9, 10, 11], for which differential equations are the natural language. In this work, first- and second order Ordinary Differential Equations (ODEs) and first- and second order Partial Differential Equations (PDEs) will be considered. To solve such differential equations, some initial conditions (and/or boundary conditions for PDEs) need to be specified. These are referred to as initial value problems. In the case of an ODEs we impose that $y(t) : [a_t, b_t] \to \mathbb{R}^d$; and in the case of PDEs we impose that $y(x,t) : [a_x, b_x]^n \times [a_t, b_t] \to \mathbb{R}^d$. If the underlying structure in a differential equation is (partially) unknown, then numerical techniques are not easily applicable since they require complete knowledge of the system (model driven). System identification methods try to derive the underlying dynamics from observations [12]. Predicting the behaviour of a system when the underlying dynamics are not identical between realizations is a difficult problem. Such problems are much more general in nature, and can be denoted as:

$$\begin{cases} f(y(t), t, d_t y, d_t^2 y) = u(y,t) \\ y(a) = y_a \\ d_t y(t)|_b = y'_b \end{cases} \tag{1}$$

for ODEs, and:

$$\begin{cases} f(y(x,t), x, t, \partial_t y, \partial_x y, \partial_t^2 y, \partial_x^2 y, \partial_t \partial_x y) = u(y,x,t) \\ y(x,a) = y_a(x) \mid y(a,t) = y_a(t) \\ y'(x,b) = y'_b(x) \mid y'(b,t) = y'_b(t) \end{cases} \tag{2}$$

in the case of PDEs. In these problem settings, we require that $u(\cdot) \in V$ a compact subset of $C[a_t, b_t]$, $C[a_x, b_x]^n$, or $C[a_y, b_y]^d$; the last constraint is hard to enforce, and therefore will be ignored in experimental situations. In the original DeepONet paper, only functions $u(\cdot) \in C[a_t, b_t]$ or $C[a_x, b_x]^n$ were considered [8], which

[1] For any order of differential equation.

we will expand upon by studying the generalized problem defined above. We established that $y(\cdot)$ is dependent on $u(\cdot)$, which cannot be learned via traditional means, which is why we require operators in this context, as we wish to learn the solution y no matter the choice of $u(\cdot)$. An operator $G : X \to Y$ takes in a space X and produces a space Y. Usually, these spaces are function spaces (or infinite sequences). Moving forward, an operator is a map defined on a compact set $V \subset C(K_1)$, where $C(K_1)$ are all continuous functions defined on some compact set of a Banach space $K_1 \subset X$, where X is the Banach space in question. The operator $G : V \to C(K_2)$ maps the compact space V into another space of all continuous functions $C(K_2)$ of a compact space $K_2 \subset Y$.

Neural Operators

We stated, in an abstract sense, that Neural Operators require a function $u(\cdot)$ as input. Function spaces are infinite dimensional vector spaces, and therefore cannot be directly trained on. In the approximation theorem below it will be shown that only a finite-dimensional sampling of this infinite-dimensional space is necessary. These sampling points are called sensor locations, and the corresponding function values the sensors.

Theorem 1 (The Universal Approximation Theorem for Operator (UATO) [13])**.** *For compact sets $K_1 \in X$, $K_2 \in \mathbb{R}^d$, $V \in C(K_1)$, a non-linear operator $G : V \to C(K_2)$ between function spaces V and $C(K_2)$ can be approximated arbitrarily close by some Neural-Network-like structure. For any $\epsilon > 0$, there are some Tauber-Wiener functions $f^{(B)}(\cdot)$, $f^{(T)}(\cdot)$, features $s \in K_1$, and parameters $\theta_W^{(R)}$, $\theta_W^{(B)}$, $\theta_b^{(B)}$, $\theta_W^{(T)}$, and $\theta_b^{(T)}$[2], such that:*

$$\big|G(u(s))(t) - \sum_{i=1}^{p}\sum_{j=1}^{m}\theta_{j(W)}^{i(R)} f^{(B)}\big(\sum_{l=1}^{r}\theta_{jl(W)}^{i(B)}u(s_l) + \theta_{j(b)}^{i(B)}\big) \quad (3)$$
$$f^{(T)}\big(\theta_{(W)}^{i(T)}t + \theta_{(b)}^{i(T)}\big)\big| < \epsilon$$

holds for all $u(\cdot) \in V$ and $y \in C(K_2)$.

The features s are what we referred to as 'sensor locations' with sensors $u(s)$. There is a sum over p that does not appear in the approximation theorem for neural networks, effectively increasing the rank of the output tensors. Conceptually, this approximation theorem states that there are some basis functions $\psi(\cdot)$ and $\phi(\cdot)$ such that for a d-dimensional $y(t)$:

$$y_i(t) = G_i(u(s))(t) \quad (4)$$
$$\approx \mathscr{G}_i(u(s))(t) = \sum_{j=1}^{p}\sum_{k=1}^{m}\theta_{ijk}\psi_{ijk}(u(s))\phi_{ij}(t). \quad (5)$$

This formulation reduces to a function basis expansion if $\psi_{ij}(u(s)) = a_i$. The parametrization $\psi(\cdot)$ can be viewed as the input basis function, and $\phi(\cdot)$ the output basis function. Motivated by this approximation theorem, operator learning can now be defined as:

$$\operatorname{argmin}_\theta \mathbb{E}_{u(\cdot),t\sim\mu_u,\mu_t}\{\|G(u(\cdot))(t) - \mathscr{G}_\theta(u(\cdot))(t)\|_{K_2}^2\} \quad (6)$$
$$= \operatorname{argmin}_\theta \Big\{\frac{1}{N}\sum_i \|G(u_i(\cdot))(t_i) - \mathscr{G}_\theta(u_i(\cdot))(t_i)\|_{K_2}^2\Big\}$$
$$= \operatorname{argmin}_\theta \left\{\frac{\sum_i \|G(u_i(\cdot))(t_i) - \mathscr{G}_\theta(u_i(\cdot))(t_i)\|_{K_2}^2}{N\mathbb{E}\{G(u(\cdot))(t)\}^2}\right\},$$

where $u(\cdot)$ and t were drawn from their respective probability measures μ_u and μ_t. The last step[3] was used to introduce the Normalized Mean Squared Error $\mathcal{L}$, a scale-invariant measure useful to compare model performance between problem settings. Moving further, operator samples comes in triplets $\{(t_i, u_i(s), y_i) \mid i \in [1, N]\}$ where N is the amount of samples, $t \in \mathbb{R}$, and $y \in \mathbb{R}^d$. The input function in-and output dimension depend on the situation. Because the proof is not constructive regarding the sensor locations, in the rest of this work these sensor locations will be uniformly drawn from some predefined region of the function space of interest. There are r sensor locations of dimension v which are mapped to a dimension w, implying that $u_i(s) \in \mathbb{R}^{r\otimes w}$. In practice we shall unroll the sensor data by letting $u(s) \in \mathbb{R}^{N\otimes r\otimes w} \to u'(s) \in \mathbb{R}^{N\otimes r'}$, where $r \otimes w \to rw \equiv r'$. In further discussion, we shall omit the prime notation. The first Neural Operator is called the *Deep Operator Network* (DeepONet) [8]. The architecture is designed to fulfil the requirements set by theorem 1.

$$\mathscr{G}_i(u(s))(t) = \sum_{j=1}^{p}\mathscr{B}_{ij}(u(s))\mathscr{T}_{ij}(t) + \theta_i^{(1)} \quad (7)$$

The operator network consists of two fully connected Neural Networks which output $d \otimes p$-sized tensors per time point[4], one for the sensors $u(s)$ which is called the *branch* network $\mathscr{B}(u(s))$, and one for the input t which is called the *trunk* network $\mathscr{T}(t)$. The above configuration is called an 'unstacked' DeepONet, there is also a 'stacked' version where every index in p is now a different Neural Network and produces a rank-2 tensor, instead of a network that produces a rank-3 tensor. The problem persisting in Neural Network training time is now more pronounced since there are two larger networks. Therefore, it might be interesting to design a novel Neural Operator that uses the concepts of Extreme Learning to speed up training, with the trade-off that prediction times are potentially slower. Since the inception of the field, many different Neural Operators have been developed; most notably integral-operators where the kernel parametrization technique varies between architecture [14]. Some notable examples are the Fourier Neural Operator (FNO) [15], which relies on the representation of a convolution in Fourier Space, the Laplace Neural Operator (LNO) [16], which uses the pole-residue form of a Laplace transform, which is suited for transient responses. Because the DeepONet is a flexible architecture, in the sense that it does not specifically require feedforward Neural Networks in the branch and trunk networks, there exist many variations [17, 18, 19, 20]. No ELM DeepONet has yet been proposed.

Extreme Learning Machines

Neural Networks are slow to train; they require many function calls for the optimizer to converge to a minimum. Extreme Learning can be achieved in one step because it relies on Linear Regression. Extreme Learning Machines (ELMs) [21] are very wide 1-layer Neural Networks that are randomly initialized and left untrained. The only trainable parameters reside in the readout, which is linear and therefore trainable by Linear Regression (or a variation). The architecture takes the following form:

$$\mathscr{N}_E(t) = f(tW + b) \quad (8)$$
$$\hat{y}(t) = \mathscr{N}_E(t)\theta^{(0)} + \theta^{(1)}.$$

[2] R, T, B is a naming convention which will become relevant in the discussion of the DeepONet

[3] $\mathbb{E}\{G(u(\cdot))(t)\}$ is a constant, and therefore has no impact on minima locations.

[4] This can be achieved be producing a dp-size output and then rolling into $d \otimes p$.

In this work W is a uniformly sampled matrix $W \sim \mathcal{U}(-\sigma/\sqrt{\dim(t)}, \sigma/\sqrt{\dim(t)})$, b is a normally sampled vector $b \sim \mathcal{N}(0, \sigma/\sqrt{\dim(t)})$. The type of distribution the values should be sampled from depends on the situation; normal and uniform distributions are seen as the standard. One must carefully select σ for every specific situation. Large values of σ are ideal for complex problems, and small values for less complex (almost linear) problems [22]. As mentioned before, the only trainable parameters are $\theta^{(0)}$ and $\theta^{(1)}$. This network lifts the input data to a latent output dimension m, also known as the width of the Extreme Learning Machine. This strategy is similar to a basis expansion, where the basis is now a combination of random transformations of the input data. It has been shown that random network models can be used to learn the dynamics of a system; there are two distinct methods to learn dynamics, namely traditional solution learning $\mathscr{N} : t \to y(t)$ [23, 24], which will be the focus of this work, and recurrent map learning $\mathscr{N} : y(t) \to y(t+\Delta t)$, which can either be a sliding-window approach [25], or Reservoir Computing (random Recurrent Neural Network) [26, 9]. In this work, we will briefly touch upon recurrent learning in the context of out-of-distribution generalization. ELMs are typically much higher in dimension than the input dimension to ensure good accuracy. In the case of functional data, one can assume that in the high-dimensional sensor data there is clearly a lower-dimensional manifold whereon the data lies [27]. We can also reasonably assume that, if there are enough sensor locations, that there will be redundant features (will be shown in section 5.3), therefore one can artificially reduce the dimensionality of the input without removing features by imposing a sparsity on W. In this work, W will be initialized such that each latent dimension feature is connected to c input features, which can easily be imposed on the matrix W by setting $r - c$ random indices i to 0 for every j in W_{ij}, where r is the input dimension. The main drawback of Extreme Learning is that the performance is dependent on the magnitude of W, b, and c, which are hyperparameters and therefore have to be fine-tuned outside of the training loop. Extreme Learning Machines can sometimes fail to match the performance of Neural Networks when the function is too complex since it can only be one layer deep. Therefore, ELMs should only be deployed when the function to be learned is reasonably learnable by a 1-layer Neural Network. Another problem ELMs face is their tendency to overfit due to their large output dimension. The most popular solution is to use Ridge Regression [28] instead of Linear Regression, which can be defined as:

$$\underset{\theta}{\operatorname{argmin}} \mathbb{E}\{\|y(t) - [\mathscr{N}_E(t)||1]\theta^{(0)} - \theta^{(1)}\|^2 + \lambda\|[\theta^{(0)}|\theta^{(1)}]\|^2\} \tag{9}$$

$$\to [\theta^{(0)}|\theta^{(1)}] = ([\mathscr{N}_E(t)||1]^t[\mathscr{N}_E(t)||1] + \lambda\mathbb{I})^{-1}[\mathscr{N}_E(t)||1]^t y(t) \tag{10}$$

where $[\cdot|\cdot]$ refers to a concatenation over the first index and $[\cdot||\cdot]$ over the second index. λ provides a penalty (regularization) on the l_2-norm of the parameters, penalizing overuse of $\mathscr{N}(t)$ features. Determining the optimal λ to optimally reduce the generalization error cannot be done internally, and it is therefore a hyperparameter. This parameter can have a large impact on the performance and should always be optimized for the problem setting by selecting the parameter that resulted in the lowest validation loss. The validation dataset will in general be a $\sim$ 20% split of the training set (in this work as well). Gradient methods for finding the optimal λ are not easily introduced; a heuristic approach will be implemented in this work. Brent's method [29] is a popular hybrid/heuristic scalar minimization algorithm based on inverse quadratic interpolation, the bisection method, and the secant method; it has been pre-implemented in many scientific coding libraries (Scipy for Python). A major reason for the increase in Neural Network popularity is the possibility for GPU accelerated gradient-based optimization. Linear-and Ridge Regression can also benefit from GPU acceleration (to a lesser extent).

3 Methodology

The DeepONet in its standard form unfortunately does not permit Extreme Learning because one cannot apply Linear/Ridge Regression to $\mathscr{B}_E$ and $\mathscr{T}_E$ separately. Therefore, we suggest a new Neural Operator that does allow Extreme Learning Machines to be used. To construct such an architecture, eq. (4) can be used as a template, which we will manipulate into a form that is easily transformable into a DeepONet; afterwards, we shall rework it to be Linear-Regression-compatible. To achieve the DeepONet form, we algebraically manipulate eq. (4) in the following way:

$$y_i(t) = G_i(u(s))(t) \approx \sum_{j=1}^{p}\sum_{k=1}^{m} \theta_{ijk}\psi_{ijk}(u(s))\phi_{ij}(t) \tag{11}$$

$$= \sum_{j=1}^{p}\sum_{k=1}^{m} \theta_{ijk}\psi_{ijk}(u(s))\phi_{ijk}(t) \tag{12}$$

$$= \sum_{l=1}^{mp} \theta_{il}\psi_{il}(u(s))\phi_{il}(t), \tag{13}$$

where we defined $\phi_{ijk}(t) = \phi_{ij}(t) \mid \forall k$, and afterwards flattened the rank-3 tensors into rank-2 tensors. If we ignore the constraint set on ϕ, therefore letting every k have a different output basis function, add a bias term, and absorb θ into the functions, then we retrieve the DeepONet if the functions are (deep) Neural Networks. The logical leap we made is that the constraint purely reduces the possible hypothesis space, therefore by ignoring it we effectively reduce bias and increase the variance of the model according to the *Bias-Variance trade-off* (BVT). More recent work on the matter has shown that generalization is not per se a zero-sum game as implied by the BVT , but far complexer in (deep) Neural Networks [30, 31]. To be able to further generalize the model to handle Linear Regression, we define[5] $\xi(u(s))(t) \equiv \psi(u(s)) \odot \phi(t)$, and flatten $\epsilon_{il} \to \epsilon_{l'}$. This requires $\theta \to \theta'$, which is a square matrix of dimension $d \otimes d$ with $mp \otimes 1$ parameter matrices on the diagonal, and $0^{mp\otimes 1}$ on the off-diagonal:

$$(\xi(u(s))(t)\theta')_i = \sum_{l=1}^{mp} \theta_{il}\psi_{il}(u(s))\phi_{il}(t) \tag{14}$$

By assuming that the basis functions can be approximated via Extreme Learning Machines $\psi(\cdot), \phi(\cdot) \to \mathscr{B}_E(\cdot), \mathscr{T}_E(\cdot)$, ignoring the conditions that were set on θ' (renaming it $\theta^{(0)}$), and adding a bias term $\theta^{(1)}$, then one retrieves the novel *Extreme Operator Network* (ExtremONet):

$$\begin{aligned} \mathscr{B}_E(u(s)) &= f^{(B)}(u(s)W^{(B)} + b^{(B)}) \\ \mathscr{T}_E(t) &= f^{(T)}(tW^{(T)} + b^{(T)}) \\ \mathscr{G}(u(s))(t) &= (\mathscr{B}_E(u(s)) \odot \mathscr{T}_E(t))\theta^{(0)} + \theta^{(1)} \end{aligned} \tag{15}$$

which is now trained via Ridge Regression, as defined in section 2, but now in the context of operator learning. This novel model has 4 hyperparameters, namely trunk scale σ_t, trunk connectivity c_t,

[5] $\odot$ Is the Hadamard product.

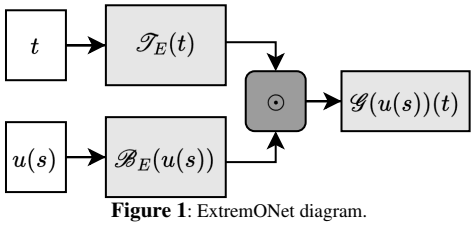

Figure 1: ExtremONet diagram.

branch scale σ_b, and branch connectivity c_b. Because time is one-dimensional there is not option but $c_t = 1$. The reconfiguration of the DeepONet from a Machine Learning view can be seen as removing even more bias from the architecture, and the possible introduction of more variance. We effectively allowed for Linear Regression by overparametrizing the UATFO because we ignored the conditions of $\theta^{(0)}$. This might have negative effects on the generalization performance of the derived model. Fortunately Ridge Regression already functions as a generalization error minimization technique; now it will handle generalization error caused by the extreme width, and the overparametrization with respect to the DeepONet. Note that the ExtremONet does not stray further from the UATO than the DeepONet; the ExtremONet is more overparametrized, but more closely aligns with the functional structure than the (possibly) layered architecture of a DeepONet.

4 Experimental details

4.1 ODE systems

Moving forward, parametrized systems refer to initial value problems where $u(\cdot) \sim u_\alpha(\cdot)$ is dependent on some finite (a) number of parameters α creating a compact set $V_\alpha \subset C(K_1)$ with $\alpha \in A \subset \mathbb{R}^a$. For instance, the polynomials: $u_\alpha(t)_i = K^k_\alpha[t] = \alpha_0 + \alpha_1 t + \alpha_2 t^2 + ... + \alpha_k t^k$. If we consider $K^k_\alpha[y]$, then the polynomials of order k generate $a = \sum_{j=0}^{k} \binom{d+j-1}{j} = \binom{d+k}{k}$ parameters, which quickly become intractable. For testing, we shall consider $d_t y = u(y)$ where $u \in K^2_\alpha[y]$, namely the Lorenz-63 system [32]:

$$\begin{aligned} d_t y_0 &= \alpha_0(y_1 - y_0), \\ d_t y_1 &= y_0(\alpha_1 - y_2) - y_1, \\ d_t y_2 &= y_0 y_1 - \alpha_2 y_2. \end{aligned} \tag{16}$$

We shall consider this system in chaotic and non-chaotic regimes. Additionally, we will also learn operators of non-parametrized function spaces like Gaussian Random Fields (GRFs) [33] . These fields are stochastic in nature and are generated by a multivariate Normal distribution with a kernel covariance term:

$$u_t \sim \mathcal{N}(0, k_l(t,t)). \tag{17}$$

The covariance matrices are generated by the kernel according to: $k_{ij}(t,t) = k(t_i, t_j)$. In practice, these GRFs are turned into functions by applying some form of interpolation (usually cubic) of some finite sampling of u. In this work, the radial basis function $k^l_{ij}(t,t) = e^{-\|t_i - t_j\|_2^2/2l^2}$ will be used as kernel. It is also possible to mix compact spaces via $k^l(y,t)$, which will not be considered in this work. The solutions generated by the Lorenz-63 parametrization will be much harder to learn since small changes in α will not always[6] result in small changes in $y(t)$. For the temporal function space, a driven pendulum will be considered:

$$d_t^2 y - \sin(y) = u(t). \tag{18}$$

[6] Divergence of trajectories for small changes in α in the chaotic regime.

ODE datasets

The Lorenz-63 training-and testing set were generated identically, but the test set contained 10x more samples. The training set consisted of 1000 realizations of 10 time points uniformly sampled from $t \in [0,1]$. The parameters were drawn uniformly from $\alpha_0 \in [0,10]$, $\alpha_1 \in [0,28]$, and $\alpha_2 \in [0, 8/3]$. There were 40 3-dimensional sensor locations uniformly drawn between $y \in [-30,30]^3$. The initial position was normally sampled once from $y_0 \sim \mathcal{N}(0,1)$. The driven pendulum training-and testing set were generated identically, but the test set contained 10x more samples. The training set consisted of 1000 realizations of 10 time points uniformly sampled between $t \in [0,1]$. l was set to be 0.1. There were 120 1-dimensional sensor locations uniformly drawn from $y \in [0,1]$. The initial condition was normally sampled once from $y_0, y_0' \sim \mathcal{N}(0, 0.1)$.

4.2 PDE systems

We shall treat operators in the context of PDEs in a similar manner. In this work, the main focus will be the parametrized 1-dimensional Sine-Gordon equation [34] and a 1-dimensional diffusion equation:

$$\partial_t^2 y - \partial_x^2 y = u_\alpha(y), \tag{19}$$

and

$$\partial_t y + \partial_x^2 y = u(x). \tag{20}$$

where the function spaces of interest are respectively: $u_\alpha(y) = \sum_{i=1}^{q} \sin(\alpha_i \pi y)$ and $u(x) \sim \mathcal{N}(0, k_l)$.

PDE datasets

The Sine-Gordon training-and testing set were generated identically, but the test set contained 10x more samples. The training set consisted of 1000 realizations of 10 time points uniformly sampled from $t \in [0,3]$. The spatial coordinates were treated via finite differences, with two equidistantly sampled grids which consisted of 100 points with $x \in [0,1]$ that were concatenated together, where the first 100 features are $y(x,t)$ and the second 100 features are $\partial_t y(x,t)$. The parameters were drawn uniformly from $\alpha \in [0,1]^3$. There were 120 1-dimensional sensor locations uniformly drawn from $y \in [-10,10]$. The initial position was a radial basis function with $y_0(x) = k_{0.1}(1/\sqrt{2}, x)$ and $y_0'(x) = 0$. A toroidal boundary condition was imposed. The diffusion training-and testing set were generated identically, but the test set contained 10x more samples. The training set consisted of 1000 realizations of 10 time points uniformly sampled from $t \in [0,1]$. The spatial coordinates were treated via finite differences, with one equidistantly sampled grid which consisted of 100 points with $x \in [0,1]$. l was set to be 0.1. There were 120 1-dimensional sensor locations uniformly drawn from $y \in [0,1]$. The initial position was a radial basis function with $y(x) = k_{0.1}(0.75, x)$. The boundary condition was a Dirichlet condition with $\partial\Omega = 0$.

4.3 Experimental setup

Because execution time will be used as a test metric, the experiment details will have to be disclosed. The tests were executed on Windows 11 Pro in a python 3.11 environment using cudaNN version 9.7 and Pytorch version 2.5.1 on a desktop equipped with an AMD

Ryzen 7 7700[7] and an Nvidia RTX 4070 Ti SUPER[8]. All mathematical operations were performed on the GPU if possible; this includes Ridge Regression. The differential equations were solved using the solve_ivp (Runge-Kutta 5(4) [35]) function in Scipy version 1.15.1; spatial coordinates were treated with a finite differences method in the case of PDEs. In some experimental situations, hyperparameters have to be tuned automatically to find the best possible performance of an ExtremONet. To do so, Bayesian optimization will be used as implemented by Scikit-Learn version 1.6.1.

5 Results

In this section, it will first be shown that the ExtremONet outperforms the DeepONet on 2/4 problems without using a hyperparameter search to minimize computation time. A statistically relevant exploration of the design parameters will be done to indicate the benefits and shortcomings of the ExtremONet. This paper will conclude with a discussion about out-of-distribution generalization in the context of operator learning, specifically conceptual drift in dynamical systems.

5.1 General comparison

A small-, medium-, and large DeepONet was compared against a small-, medium-, and large ExtremONet. The models were trained on 10 000 total samples. The DeepONets were 1-layer 100-wide, 2-layers 200-wide, 3-layers 500 wide , with a p-size of 10, 10, and 20 respectively; trained with the AdamW optimizer [36] with a learning rate of $2\cdot10^{-2}$, $5\cdot10^{-3}$, and $1\cdot10^{-3}$ and weight decay of 10^{-5} until convergence. In this work we will define convergence via a stopping condition; training will stop early if the following condition is no longer met:

$$\frac{1}{k}\log_{10}\left(\Pi_{i=0}^{k-1}\frac{\mathcal{L}_{n-i}^{val}}{\mathcal{L}_{n-k-1-i}^{val}}\right) < -\epsilon. \tag{21}$$

The training loop will end when the magnitude of the slope of the current validation loss is less than ϵ, which will be set to 0 and $k = 500$. The ExtremONets were 100-, 300-, and 1000-wide respectively with $\sigma_t = 10$, $\sigma_b = 0.1$, and $c_b = 1$, which will be the standard parametrization unless specified otherwise. λ was found via Brent's algorithm, ran for 100 iterations, or until $\Delta\lambda < 10^{-2}$. No hyperparameter selection was executed; all were initialized identically. All models will use a hyperbolic tangent as activation function; the DeepONets will have a linear readout. In fig. 2 it is shown that the ExtremONet outperformed the DeepONet on the pendulum- and Sine-Gordon dataset. The DeepONet significantly outperformed the ExtremONet on the Lorenz-63 and slightly on diffusion dataset. Another important behavioural distinction is the generalization error throughout their respective training iterations. The ExtremONets validation error was almost perfectly correlated to the training error, which was not the case for the large DeepONet, which started overfitting on all problem settings. The models are tied regarding the training error; the picture changes drastically when training time is taken into account. In fig. 3 it was shown that the ExtremONets took between 100-10 000x less time to complete training, whilst keeping the prediction time almost identical (<10x). Because Ridge Regression is a solved optimization problem, on top of the short runtime, it is guaranteed that the found solutions are the global minima.

[7] Overclocked to 5.4GHz.
[8] +150MHz Core- and +1250MHz memory overclock.

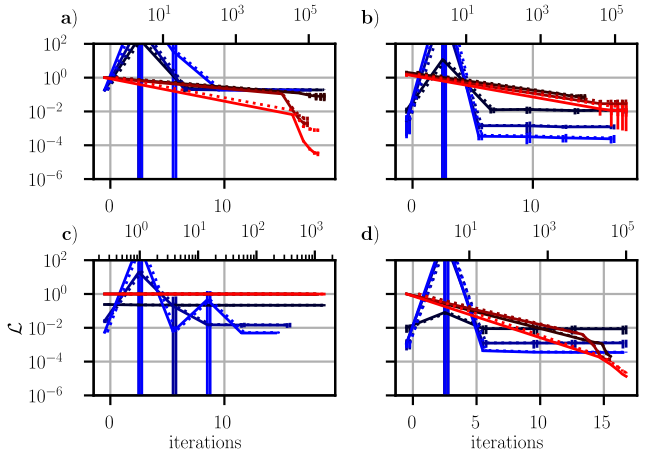

Figure 2: Train- and validation loss history (filled and dotted line) of three DeepONet (red) and three ExtremONets (blue). Model size is tied to colour shade. The upper x-axis represent DeepONet training iterations, lower x-axis represents Ridge Regression regularization parameter search iterations. The y-axis is $\log_{10}$-scale NMSE. **a)** Lorenz-63 system. **b)** Gravity pendulum ODE with GRF (t). **c)** Parametrized Sine-Gordon PDE. **d)** Diffusion PDE with GRF (x).

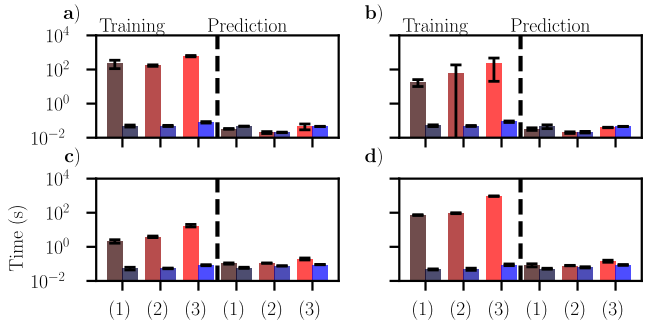

Figure 3: Mean train- and prediction time of three DeepONet (red) and three ExtremONets (blue). Model size is tied to colour shade. The y-axis is $\log_{10}$-scale seconds. **a)** Lorenz-63 system. **b)** Gravity pendulum ODE with GRF (t). **c)** Parametrized Sine-Gordon PDE. **d)** Diffusion PDE with GRF (x).

5.2 Width dependence

The most limiting behaviour of the ExtremONet is a plateau in the train- and test loss with respect to the model width. This behaviour is expected because the model is selected to minimize validation loss; therefore, overfitting is almost impossible, and the lack of depth in the model will negatively affect its performance on complex problems. The train error-, test error-, train time-, and prediction time width dependence were approximated by a power-law function for every system, and then propagated to find an average result. The power-law fit returned:

$$\mathcal{L}_{\text{train}}(m) \sim m^{-0.98}, \quad \mathcal{L}_{\text{test}}(m) \sim -m^{-0.96},$$
$$t_{\text{train}}(m) \sim m^{1.09\pm0.30}s, \quad t_{\text{predict}}(m) \sim m^{0.97\pm0.04}s.$$

The run time is not corrected for the time-delay related to moving information between CPU and GPU, which becomes less relevant when the model size increases. The large variance on the train time is due to the stopping condition of Brent's algorithm.

5.3 Sensor dependence

Sensor data is 'experimentally expensive' in a sense that this non-grid-invariant measurement of the input function might not be easy to realize. This necessitates the models to make maximal use of the sensor data. In fig. 4 it was shown that the Lorenz-63-, pendulum-, and diffusion systems needed between $\sim 5-10$, $\sim 15-50$, and $\sim 20-100$ sensors to saturate the performance of the 1000-wide

ExtremONet. This is in contrast to the original DeepONet publication where a similar pendulum problem, solved over the same time period and dependent on the same input space, required on average 10 sensors. This might imply that the ExtremONet sensor-efficiency is highly dependent on its random initialization. The Lorenz-63 problem setting was almost independent of the number of sensors, indicating that it was too hard to learn for the ExtremONet. A less obvious problem appears when there are few sensors. Due to the sensor locations being randomly initialized, it might be that those locations do not carry enough information about the lower-dimensional representation of the input function for the ExtremONet/DeepONet to learn. Therefore, sensor dependence cannot be studied in depth without first determining optimal sensor locations (for which there is currently no optimal procedure). One can make an educated guess based on information theory; we expect that higher variance regions carry more information, which can be optimized in the data collection phase. ExtremONets, due to their short training time, could alternatively be used to heuristically find these ideal sensor locations. This feat was already accomplished using DeepONets [37], which could be vastly sped up.

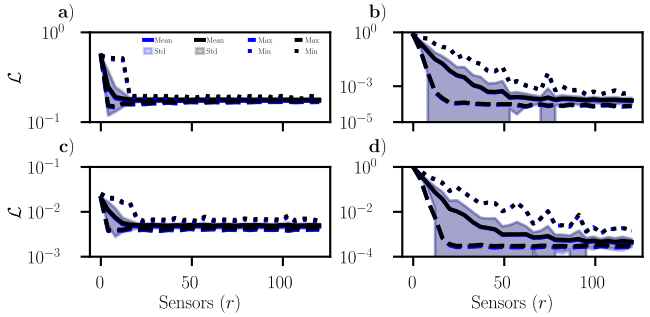

Figure 4: ExtremONet train loss (blue) and test loss (black) mean, standard deviation, maximum, and minimum with respect to the amount of sensors (100 repeats). The y-scale is $\log_{10}$-scale NMSE. **a)** Lorenz-63 system. **b)** Gravity pendulum ODE with GRF (t). **c)** Parametrized Sine-Gordon PDE. **d)** Diffusion PDE with GRF (x).

5.4 Function space complexity & out-of-distribution learning for PDEs

Due to the recent influx of performant domain-specific architectures, discussions about out-of-distribution generalization have become more prevalent for robust real-world deployment [38, 39, 40, 41]. Many problem settings do not guarantee that the test data is sampled from the same distribution as the train data, also known as Independent and Identically Distributed (IID), for example in cases where the test data is not available at time-of-training (such as online learning, where the test data arrives in streams). The field is still in its infancy; [42] provides an overview of OOD generalization research. One of the categories of OOD discussed is conceptual drift, where the relation $X \sim Y$ drifts. Because Neural Operators learn maps between function spaces, one can divide conceptual drift of dynamics into two classifications:

- **In-function-space dynamical drift** occurs when the dynamics drift in such a way that it still remains in the input function space sampled by the training data. This drift occurs for function approximators, and potentially for Neural Operators when the drift rate is fast enough.
- **Out-of-function-space dynamical drift** occurs when the dynamics drift in such a way that it moves beyond the input function space sampled by the training data. This drift only occurs applies to operators.

The first classification would manifest itself as $F_{u(x)} : x, t \rightarrow y(x,t)$ in the case of PDEs, where there is now an unknown shift of $u(x)$ present in the dynamical equation $F(x,t)$. This can be translated to an unknown time dependence in the underlying differential equations $u(x) \sim u_t(x,t)$. Because Neural Operators measure the input function space[9] at the initial time, they do not have information about drifts that occur afterwards. This can be resolved by moving the model to phase space, in a similar manner to RNNs (without memory)[10]:

$$\mathcal{G}_{\Delta t} : u(s), y(t) \rightarrow y(t + \Delta t), \tag{22}$$

which creates a trajectory via:

$$y(n\Delta t) = (\circ_{k=1}^{n} \mathcal{G}_{\Delta t}(u(s))(\cdot)) \circ y(0). \tag{23}$$

$u(s)$ can be adjusted at each composition, therefore allowing for rapid tuning. In the case of drift, one would have:

$$y(n\Delta t) = (\circ_{k=1}^{n} \mathcal{G}_{\Delta t}(u(s, k\Delta t))(\cdot)) \circ y(0). \tag{24}$$

Notice that the function space drift has been resolved. There is however an implicit drift present in the phase space. Because the dynamics shift throughout the trajectory, it will reach locations in the phase space not visited for the drifted input function (at each time point) starting from the general initial condition. For the ExtremONet in phase space to generalize to such a test set, it needs to learn the map beyond the locations visited by a trajectory, therefore truly learning the underlying map and not just interpolating the data. To test this, one needs to verify whether an ExtremONet in phase space can account for drifts moving trajectories of PDEs to locations not seen in their respective function space realizations during training. This will be done by training an ExtremONet in phase space, and then testing the performance on trajectories with drifting GRFs. These will be constructed via spherical transitions between two GRF samples:

$$u_{drift}(x,t) \sim \sqrt{d\frac{t}{t_f}}u_0(x) + \sqrt{1 - d\frac{t}{t_f}}u_1(x), \tag{25}$$

where $d \in [0,1]$ is the drift factor, and t_f is the final time point of the data. Because u_1 and u_2 have the same covariance matrix (defined by k_l), a normalized root combination will again be a sample from GRF with k_l:

$$\begin{aligned} &\mathrm{Cov}(u_{drift}(x,t), u_{drift}(x',t)) && (26)\\ &= d\frac{t}{t_f}k_l(x,x') + (1 - d\frac{t}{t_f})k_l(x,x') && (27)\\ &= k_l(x,x'), && (28) \end{aligned}$$

where we used the fact that the covariance between two samples is zero because they are independent. To test whether an ExtremONet in phase space can learn dynamical systems of various complexity, and is robust against in-function-space dynamical drift; a 1000-wide ExtremONet in phase space with $\sigma_t = 0.1$, $c_t = 1$, $\sigma_b = 0.1$, and $c_b = 1$ was trained on various datasets. For every GRF length scale (and drift scale for the test set), 1000 diffusion systems with spatial GRF were generated, where 100 equidistant time points with $t \in [0,1]$ were sampled. Each test was repeated 30 times; the results are found in fig. 5. The (almost) independence on the drift parameter after a certain level of training data complexity indicates that

[9] Not per se all of the differential equation

[10] This setup is just a proof of concept, a more robust method such as a Reservoir Computing trunk network should be considered.

dynamical drift can be accounted for by re-measuring the input function between time points. It is important to note that this does not occur for less complex training sets, which is most likely because such systems do not have enough variety in the dynamics, resulting in phase space regions with low trajectory density and regions with high trajectory density. These low density regions hinder the ExtremONet in phase space to generalize. The second classification

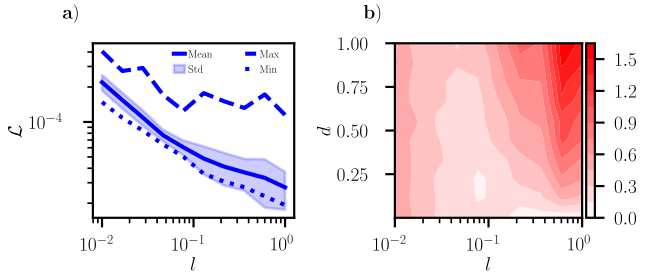

Figure 5: ExtremONet in phase space, trained on a diffusion system with spatial GRF; 30 repeats. **a)** Train loss mean, standard deviation, maximum and minimum with respect to the GRF length scale $l \in [0.01, 1]$, The y-scale is $\log_{10}$-scale NMSE. **b)** $\log_{10}(\mathcal{L}_{test}/\mathcal{L}_{train})$ with respect to the train- and test GRF length scale ($l \in [0.01, 1]$) on the x-axis, and the test drift amount ($d \in [0, 1]$) on the y-axis, The colorbar is $\log_{10}$-scale NMSE.

more closely aligns to the definition of conceptual drift (for operators). To study this, one has to check whether the ExtremONet can generalize outside of its learned function space to ensure that the network can predict accurately regardless of drift. This will be done by training and testing the ExtremONet on pendulum- and diffusion system, driven by GRFs of different length scales[11] with respect to one another. Training on different GRF length scales also allows us to measure the impact of the complexity of an input function space on the performance of an ExtremONet as well. To test whether an ExtremONet is robust against out-of-function-space dynamical drift, for every GRF length scale 1000 pendulum- and diffusion systems with temporal- and spatial GRF were generated, where 10 uniformly distributed time points with $t \in [0, 1]$ were sampled. This test also allows us to verify the performance of the ExtremONet on various levels of function space complexity. Each test was repeated 30 times. The results are found in fig. 6 and fig. 7. The ExtremONet was able to generalize to larger length scale GRF systems, indicating that its knowledge from complexer systems allowed the ExtremONet to generalize from learning complex systems to predict simple systems. It was not able to generalize from low complexity training data to high complexity test data, which was to be expected.

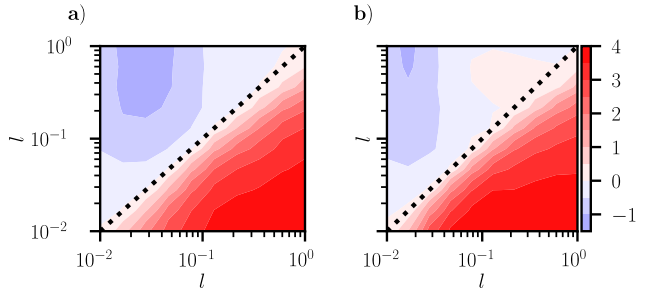

Figure 6: ExtremONet train loss mean, standard deviation, maximum, and minimum for various amounts GRF length scale ($l \in [0.01, 1]$) for the training set, The y-scale is $\log_{10}$-scale NMSE; 30 repeats. **a)** Gravity pendulum ODE with GRF (t). **b)** Diffusion PDE with GRF (x).

[11] Drift in the length scale creates a drift in the sample distribution.

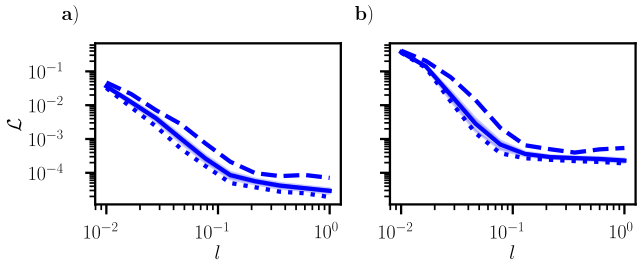

Figure 7: ExtremONet out-of-function-space dynamical drift performance, measured by $\log_{10}(\mathcal{L}_{test}/\mathcal{L}_{train})$. Train GRF scale on the x-axis, and test GRF scale on the y-axis; 30 repeats. **a)** Gravity pendulum ODE with GRF (t). **b)** Diffusion PDE with GRF (x).

6 Conclusion

In this paper, we showed that an Extreme-Learning-based variation of the DeepONet was two to four orders of magnitude faster to train, and was able to outperform large DeepONets in half of the problem settings. We demonstrated that the ExtremONet generalized much better than the DeepONet in every situation due to the validation loss selection metric of Ridge Regression. More complex situations require a multi-layered structure, which is not compatible with ELMs[12]. We also showed that the ExtremONet can generalize out-of-function-space, as long as the drift is to lower complexity; They are also robust against real-time in-function-space dynamical drift when the model is moved into phase space. The most important result in this work is the ExtremONets training time reduction relative to the DeepONet. This allows for rapid development of Neural Operators in situations where time is most valuable. In situations where the model needs to be adjusted as new data arrives, traditional DeepONets would have to rely on on-line learning [43] to adjust their parameters; ExtremONets might be fast enough to essentially retrain the model without relying on on-line learning. One can also include an on-line ELM strategy, as mentioned in [44]. On another note; as climate change, and AI's energy consumption [45, 46], becomes an ever more relevant geo-political topic, the ExtremONet also provides a responsible alternative to DeepONets. In conclusion, the ExtremONet is a robust, rapid Neural Operator, suitable for simple or moderately complex real-world problem settings, or when there is limited access to compute.

References

[1] Georgia Karagiorgi et al. Machine Learning in the Search for New Fundamental Physics, 2021. Version Number: 1.

[2] Thomas Huang et al. An Earth System Digital Twin for Flood Prediction and Analysis. In *IGARSS 2022 - 2022 IEEE International Geoscience and Remote Sensing Symposium*, pages 4735–4738, Kuala Lumpur, Malaysia, July 2022. IEEE. ISBN 978-1-6654-2792-0. doi: 10.1109/IGARSS46834.2022.9884830.

[3] John Jumper et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, August 2021. ISSN 0028-0836, 1476-4687. doi: 10.1038/s41586-021-03819-2.

[4] E. Ivo Alves. Earthquake Forecasting Using Neural Networks: Results and Future Work. *Nonlinear Dynamics*, 44(1-4):341–349, June 2006. ISSN 0924-090X, 1573-269X. doi: 10.1007/s11071-006-2018-1.

[5] Emma Strubell et al. Energy and Policy Considerations for Deep Learning in NLP, 2019. Version Number: 1.

[6] M. Raissi et al. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, February 2019. ISSN 00219991. doi: 10.1016/j.jcp.2018.10.045.

[12] There is such a concept as multi-layer ELMs [23], but they are not equivalent to multi-layer neural networks.

[7] Hrishikesh Viswanath et al. Neural Operator: Is data all you need to model the world? An insight into the impact of Physics Informed Machine Learning, 2023. Version Number: 2.

[8] Lu Lu et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3(3):218–229, March 2021. ISSN 2522-5839. doi: 10.1038/s42256-021-00302-5.

[9] Xiaodong Na et al. Physics-informed hierarchical echo state network for predicting the dynamics of chaotic systems. *Expert Systems with Applications*, 228:120155, October 2023. ISSN 09574174. doi: 10.1016/j.eswa.2023.120155.

[10] Manuel A. Roehrl et al. Modeling System Dynamics with Physics-Informed Neural Networks Based on Lagrangian Mechanics. *IFAC-PapersOnLine*, 53(2):9195–9200, 2020. ISSN 24058963. doi: 10.1016/j.ifacol.2020.12.2182.

[11] Eric Aislan Antonelo et al. Physics-informed neural nets for control of dynamical systems. *Neurocomputing*, 579:127419, April 2024. ISSN 09252312. doi: 10.1016/j.neucom.2024.127419.

[12] Gianluigi Pillonetto et al. Deep networks for system identification: A survey. *Automatica*, 171:111907, January 2025. ISSN 00051098. doi: 10.1016/j.automatica.2024.111907.

[13] Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE Transactions on Neural Networks*, 6(4):911–917, July 1995. ISSN 10459227. doi: 10.1109/72.392253.

[14] Nikola Kovachki et al. Neural Operator: Learning Maps Between Function Spaces. 2021. doi: 10.48550/ARXIV.2108.08481. Publisher: arXiv Version Number: 6.

[15] Zongyi Li et al. Fourier Neural Operator for Parametric Partial Differential Equations, 2020. Version Number: 3.

[16] Qianying Cao et al. LNO: Laplace Neural Operator for Solving Differential Equations, 2023. Version Number: 2.

[17] Jiahao Zhang et al. MultiAuto-DeepONet: A Multi-resolution Autoencoder DeepONet for Nonlinear Dimension Reduction, Uncertainty Quantification and Operator Learning of Forward and Inverse Stochastic Problems, 2022. Version Number: 1.

[18] Yixuan Sun et al. DeepGraphONet: A Deep Graph Operator Network to Learn and Zero-shot Transfer the Dynamic Response of Networked Systems, 2022. Version Number: 1.

[19] Junyan He et al. Sequential Deep Operator Networks (S-DeepONet) for predicting full-field solutions under time-dependent loads. *Engineering Applications of Artificial Intelligence*, 127:107258, January 2024. ISSN 09521976. doi: 10.1016/j.engappai.2023.107258.

[20] Diab W. Abueidda et al. DeepOKAN: Deep Operator Network Based on Kolmogorov Arnold Networks for Mechanics Problems, 2024. Version Number: 3.

[21] Guang-Bin Huang et al. Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1-3):489–501, December 2006. ISSN 09252312. doi: 10.1016/j.neucom.2005.12.126.

[22] Liliya A. Demidova and Vladimir E. Zhuravlev. The Study on Initialization Aspects of the Extreme Learning Machine Parameters by Random Values. In *2024 6th International Conference on Control Systems, Mathematical Modeling, Automation and Energy Efficiency (SUMMA)*, pages 364–369, Lipetsk, Russian Federation, November 2024. IEEE. ISBN 979-8-3315-3217-8. doi: 10.1109/SUMMA64428.2024.10803774.

[23] Gizem Atac Kale and Cihan Karakuzu. Multilayer extreme learning machines and their modeling performance on dynamical systems. *Applied Soft Computing*, 122:108861, June 2022. ISSN 15684946. doi: 10.1016/j.asoc.2022.108861.

[24] Gianluca Fabiani et al. Numerical solution and bifurcation analysis of nonlinear partial differential equations with extreme learning machines. *Journal of Scientific Computing*, 89(2):44, November 2021. ISSN 0885-7474, 1573-7691. doi: 10.1007/s10915-021-01650-5.

[25] Hans Harder et al. Solving Partial Differential Equations with Equivariant Extreme Learning Machines, 2024. Version Number: 5.

[26] Piotr Antonik et al. Using a reservoir computer to learn chaotic attractors, with applications to chaos synchronization and cryptography. *Physical Review E*, 98(1):012215, July 2018. ISSN 2470-0045, 2470-0053. doi: 10.1103/PhysRevE.98.012215.

[27] Charles Fefferman et al. Testing the Manifold Hypothesis, 2013. Version Number: 2.

[28] Guoqiang Li and Peifeng Niu. An enhanced extreme learning machine based on ridge regression for regression. *Neural Computing and Applications*, 22(3-4):803–810, March 2013. ISSN 0941-0643, 1433-3058. doi: 10.1007/s00521-011-0771-7.

[29] R. P. Brent. *Algorithms for minimization without derivatives*. Prentice-Hall series in automatic computation. Prentice-Hall, Englewood Cliffs, N.J, 1972. ISBN 978-0-13-022335-7.

[30] Brady andothers Neal. A Modern Take on the Bias-Variance Tradeoff in Neural Networks. 2018. doi: 10.48550/ARXIV.1810.08591. Publisher: arXiv Version Number: 4.

[31] Yehuda Dar et al. A Farewell to the Bias-Variance Tradeoff? An Overview of the Theory of Overparameterized Machine Learning, 2021. Version Number: 1.

[32] Edward N. Lorenz. Deterministic Nonperiodic Flow. *Journal of the Atmospheric Sciences*, 20(2):130–141, March 1963. ISSN 0022-4928, 1520-0469. doi: 10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2.

[33] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, November 2005. ISBN 978-0-262-25683-4. doi: 10.7551/mitpress/3206.001.0001.

[34] Sergei B. Kuksin and Dario Bambusi. Hamiltonian PDEs. In *Handbook of Dynamical Systems*, volume 1, pages 1087–1133. Elsevier, 2006. ISBN 978-0-444-52055-5. doi: 10.1016/S1874-575X(06)80040-8.

[35] J.R. Dormand and P.J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, March 1980. ISSN 03770427. doi: 10.1016/0771-050X(80)90013-3.

[36] Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization, 2017. Version Number: 3.

[37] Ruyin Wan, Ehsan Kharazmi, Michael S Triantafyllou, and George Em Karniadakis. DeepVIVONet: Using deep neural operators to optimize sensor locations with application to vortex-induced vibrations, 2025. Version Number: 1.

[38] Jie Lu et al. Learning under Concept Drift: A Review. 2020. doi: 10.48550/ARXIV.2004.05785. Publisher: arXiv Version Number: 1.

[39] Simon Kornblith et al. Why Do Better Loss Functions Lead to Less Transferable Features?, 2020. Version Number: 2.

[40] Pang Wei Koh et al. WILDS: A Benchmark of in-the-Wild Distribution Shifts, 2020. Version Number: 3.

[41] Dan Hendrycks and Thomas Dietterich. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations, 2019. Version Number: 1.

[42] Jiashuo Liu et al. Towards Out-Of-Distribution Generalization: A Survey, 2021. Version Number: 2.

[43] Elad Hazan. Introduction to Online Convex Optimization, 2019. Version Number: 3.

[44] Jian Wang et al. A review on extreme learning machine. *Multimedia Tools and Applications*, 81(29):41611–41660, December 2022. ISSN 1380-7501, 1573-7721. doi: 10.1007/s11042-021-11007-7.

[45] Sergio Aquino-Brítez, Pablo García-Sánchez, Andrés Ortiz, and Diego Aquino-Brítez. Towards an Energy Consumption Index for Deep Learning Models: A Comparative Analysis of Architectures, GPUs, and Measurement Tools. *Sensors*, 25(3):846, January 2025. ISSN 1424-8220. doi: 10.3390/s25030846. URL https://www.mdpi.com/1424-8220/25/3/846.

[46] Ian Wright. ChatGPT Energy Consumption Visualized.